

5 Linear programming	5.1	Formulation of problems as linear programs including the meaning and use of slack, surplus and artificial variables.	For example, $3x + 2y \leq 20 \Rightarrow 3x + 2y + s_1 = 20$ $2x + 5y \leq 35 \Rightarrow 2x + 5y + s_2 = 35$ $x + y \geq 5 \Rightarrow x + y - s_3 + t_1 = 5$ where s_1, s_2 are slack variables, s_3 is a surplus variable and t_1 is an artificial variable.
	5.2	Graphical solution of two variable problems using objective line and vertex methods including cases where integer solutions are required.	
	5.3	The Simplex algorithm and tableau for maximising and minimising problems with $\leq$ constraints.	Problems will be restricted to those with a maximum of four variables (excluding slack variables) and four constraints, in addition to non-negativity conditions.
	5.4	The two-stage Simplex and big-M methods for maximising and minimising problems which may include both $\leq$ and $\geq$ constraints.	Problems will be restricted to those with a maximum of four variables (excluding slack, surplus and artificial variables) and four constraints, in addition to any non-negativity conditions.

Glossary for Decision Mathematics 1

Algorithms and graph theory

The **efficiency** of an algorithm is a measure of the 'run-time' of the algorithm and in most cases is proportional to the number of operations that must be carried out.

The **size** of a problem is a measure of its complexity and so in the case of algorithms on graphs it is likely to be the number of vertices on the graph.

The **order** of an algorithm is a measure of its efficiency as a function of the size of the problem.

In a list containing N items the 'middle' item has position $\lceil \frac{1}{2}(N+1) \rceil$ if N is odd $\lceil \frac{1}{2}(N+2) \rceil$ if N is even, so that if $N=9$, the middle item is the 5th and if $N=6$ it is the 4th.

A **graph** G consists of points (**vertices** or **nodes**) which are connected by lines (**edges** or **arcs**).

A **subgraph** of G is a graph, each of whose vertices belongs to G and each of whose edges belongs to G .

If a graph has a number associated with each edge (usually called its **weight**) then the graph is called a **weighted graph** or **network**.

The **degree** or **valency** of a vertex is the number of edges incident to it. A vertex is **odd** (**even**) if it has **odd** (**even**) degree.

A **path** is a finite sequence of edges, such that the end vertex of one edge in the sequence is the start vertex of the next, and in which no vertex appears more than once.

A **cycle** (**circuit**) is a closed path, i.e. the end vertex of the last edge is the start vertex of the first edge.

Two vertices are **connected** if there is a path between them. A graph is **connected** if all its vertices are connected.

If the edges of a graph have a direction associated with them they are known as **directed edges** and the graph is known as a **digraph**.

A **tree** is a connected graph with no cycles.

A **spanning tree** of a graph G is a subgraph which includes all the vertices of G and is also a tree.

An **Eulerian graph** is a graph with every vertex of even degree. An **Eulerian cycle** is a cycle that includes every edge of a graph exactly once.

A **semi-Eulerian graph** is a graph with exactly two vertices of odd degree.

A **Hamiltonian cycle** is a cycle that passes through every vertex of a graph once and only once, and returns to its start vertex.

A graph that can be drawn in a plane in such a way that no two edges meet each other, except at a vertex to which they are both incident, is called a **planar graph**.

Two graphs are **isomorphic** if they have the same number of vertices and the degrees of corresponding vertices are the same.

Algorithms on graphs II

The **travelling salesman problem** is 'find a route of minimum length which visits every vertex in an undirected network'. In the '**classical**' problem, each vertex is visited once only. In the '**practical**' problem, a vertex may be revisited.

For three vertices A , B and C , the **triangular inequality** is 'length $AB \leq \text{length } AC + \text{length } CB$ ', where AB is a longest length.

A **walk** in a network is a finite sequence of edges such that the end vertex of one edge is the start vertex of the next.

A walk which visits every vertex, returning to its starting vertex, is called a **tour**.

Critical path analysis

The **total float** $F(i, j)$ of activity (i, j) is defined to be $F(i, j) = l_j - e_i - \text{duration } (i, j)$, where e_i is the earliest time for event i and l_j is the latest time for event j .

Linear programming

Constraints of a linear programming problem will be re-written as

$$3x + 2y \leq 20 \Rightarrow 3x + 2y + s_1 = 20$$

$$2x + 5y \leq 35 \Rightarrow 2x + 5y + s_2 = 35$$

$$x + y \geq 5 \Rightarrow x + y - s_3 + t_1 = 5$$

where s_1, s_2 are slack variables, s_3 is a surplus variable and t_1 is an artificial variable.

The **simplex tableau** for the linear programming problem:

$$\text{Maximise } P = 14x + 12y + 13z,$$

$$\text{Subject to } 4x + 5y + 3z \leq 16,$$

$$5x + 4y + 6z \leq 24,$$

will be written as

Basic variable	x	y	z	s_1	s_2	Value
s_1	4	5	3	1	0	16
s_2	5	4	6	0	1	24
P	-14	-12	-13	0	0	0

where s_1 and s_2 are slack variables.